2026-05-06-p1-f20-theme-system

P1-F20 — Theme System Implementation Plan (FINAL Phase 1 feature)

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (`- [ ]`) syntax for tracking.

> **Programme position:** F20 is the **last** Phase 1 feature of CLI-Agent Fusion. Task T09 close-out records "Phase 1 of CLI-Agent Fusion programme complete" in PROGRESS.md alongside the F20 summary.

Goal: Ship a real, end-to-end **5-role semantic theme system** for the HelixCode CLI agent. F20 adds an `internal/theme/` package with `Theme` / `Role` / `Color` / `ColorDepth` / `ThemeName` value types, three frozen built-in themes (`dark` / `light` / `none`), an env-var + `$COLORFGBG`-fallback theme-name resolver, a `$COLORTERM`/`$TERM`-driven color-depth auto-detector, an optional YAML override loader at `$XDG_CONFIG_HOME/helixcode/theme.yaml`, and a `Styler` that wraps text with role-coded ANSI SGR sequences (or returns text unchanged for plain mode / depth=Off / theme=none). Wires through F18's existing `RenderTextBlock` / `RenderLines` helpers via a decorator pattern — F18's `Renderer` interface is **unchanged**. Adds a read-only `/theme` slash command (`status` / `list` / `show <name>`); NO cobra subcommand. Plain-mode-zero-color is structurally enforced by overriding the Styler's depth to `DepthOff` whenever `renderer.Mode() == render.ModePlain`.

Architecture: New `internal/theme/` package with `types.go` (`Theme` + `Role` enum + `Color` + `ColorDepth` + `ThemeName` + sentinels `ErrUnknownTheme`/`ErrInvalidYAML`/`ErrInvalidRole` + `Reset` const + `numRoles` private const), `builtin.go` (concrete `ThemeDark` / `ThemeLight` / `ThemeNone` literals with full per-depth palettes per spec §3.4 + `BuiltIn()` registry function), `detect.go` (`DetectThemeName(env func(string) string) (ThemeName, ResolvedSource)` + `DetectColorDepth(env func(string) string) ColorDepth` — both pure functions of an injected env closure, no `os.Getenv` inside), `loader.go` (`LoaderOptions{Env, ConfigDir, Filesystem}` + `Loader.Load() (*Styler, ResolvedSource, error)` + `Styler{Stylize, Theme, Depth, Source, WithDepth}` + YAML merge into the `dark` built-in baseline + ResolvedSource constants `SourceEnv`/`SourceColorFGBG`/`SourceDefault`). New `internal/commands/theme_command.go` for the `/theme` slash command (`Name() == "theme"`; subcommands `status` / `list` / `show <name>`). Two existing files get tiny additions: `cmd/cli/main.go` (construct `theme.Loader` + `theme.Styler` adjacent to F18's renderer; wire the Styler into the `handleGenerate` non-stream `RenderTextBlock` call; register `/theme`); `internal/commands/registry.go` (no schema change; one new `Register(...)` call site).

Tech Stack: Go 1.26, testify v1.11, zap (already in `go.mod`), `gopkg.in/yaml.v3` (currently indirect; F20 promotes to direct at the version already in `go.sum`). **Zero new external deps.** F18's `internal/render` is reused for the rendering boundary; F19's `internal/tools/askuser` is unchanged. `go mod tidy` after T05 must produce ONLY the indirect→direct promotion line of yaml.v3 in `go.mod`; no new entries in `go.sum`. T08's verification step asserts this loudly.

Spec: `docs/superpowers/specs/2026-05-06-p1-f20-theme-system-design.md` (commit `0e97afa`).

Working directory for `go` commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F20 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/theme internal/commands/theme_command.go && echo
        BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — F20 can degenerate in four ways: (a) the package compiles, `/theme list` lists three themes, but no call site actually wraps text with the Styler, so fancy-mode output never carries an ANSI byte (theme "applied" without ANSI evidence); (b) the depth detector or loader picks `Truecolor` regardless of `$COLORTERM` value, so users on 16-color terminals see truecolor escapes as visual garbage; (c) plain-mode somehow leaks ANSI through the theme path (e.g. a regression in the `DepthOff` branch, or a call site bypassing the Styler and indexing `theme.ThemeDark.Truecolor[RoleInfo]` directly); (d) the env var `HELIXCODE_THEME` is honoured but the `$COLORFGBG`/`$COLORTERM`/`$TERM` detection branches are never exercised in tests, so a regression silently downgrades every user. The four "what counts as theme system works" criteria — (1) the production `Styler.Stylize` wraps text with the expected SGR open + reset bytes for each (role × depth) of each built-in (Phase A/B byte assertions); (2) plain-mode renderer + Styler force-overridden to `DepthOff` produces ZERO `0x1b` bytes in captured output (Phase C zero-byte invariant); (3) the depth auto-detector covers `Truecolor` / `ANSI256` / `ANSI16` / `Off` branches with synthesised env vars (Phase D); (4) YAML override loads custom palette and merges into the `dark` baseline correctly (Phase E byte equality on overridden vs inherited slots) — are each tested with both unit assertions AND a Challenge phase. The Challenge harness uses positive evidence: byte counts (Phase A: exactly 2 ANSI sequences per token), byte content + ordering (Phase B: open < literal < reset by offset), zero-byte invariants (Phase C: `bytes.IndexByte(captured, 0x1b) == -1`), depth-resolution byte-equality (Phase D), and YAML-merge byte-equality (Phase E). Byte-evidence mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.

Why this is consequential: the theme system is the visible surface every Phase 1 feature flows through. F18's renderer carries the bytes; F19's prompter formats the menu; every slash command from F09 through F19 prints text the user reads — F20 is what makes that text *legible* (errors stand out from info; highlights mark choices; dim de-emphasises hints). F20's discriminating tests are: (i) the Challenge's PHASE-A (byte-count invariant `bytes.Count(out, "\x1b[") == 2 * tokens` — proves real ANSI emission, not zero or excess); (ii) the Challenge's PHASE-C (plain-mode zero-`0x1b` invariant — proves the no-leak guarantee survives the production code path, not just the unit-tested helper); (iii) the Challenge's PHASE-D (six representative env combos resolving to the expected `(name, source, depth)` tuple — proves the detection ladder is wired correctly, not just present); (iv) the Challenge's PHASE-E (YAML override on tempdir + `LoaderOptions.ConfigDir` injection asserts custom slot wraps with custom code AND un-mentioned slot inherits from `dark` baseline — proves merge semantics work end-to-end). All four must produce positive evidence; none can be satisfied by absence-of-error.

Task list

- [ ] P1-F20-T01 — bootstrap evidence + advance PROGRESS to F20
- [ ] P1-F20-T02 — `internal/theme/types.go`: Theme + Role + Color + ColorDepth + ThemeName + sentinels + Reset const (TDD)
- [ ] P1-F20-T03 — `internal/theme/builtin.go`: ThemeDark + ThemeLight + ThemeNone literals + BuiltIn() registry (TDD; pin §3.4 byte tables)
- [ ] P1-F20-T04 — `internal/theme/detect.go`: DetectThemeName + DetectColorDepth pure functions of an injected env closure (TDD; table-drive every branch)
- [ ] P1-F20-T05 — `internal/theme/loader.go`: LoaderOptions + Loader.Load + Styler + Stylize + WithDepth + YAML merge into dark baseline (TDD; real tempdir + injected fs)
- [ ] P1-F20-T06 — Wire Styler into F18 in `cmd/cli/main.go` (handleGenerate non-stream RenderTextBlock + plain-mode WithDepth(Off) override) (TDD)
- [ ] P1-F20-T07 — `/theme` slash command (status / list / show) + main.go registration (TDD) + integration test
- [ ] P1-F20-T08 — Challenge harness: 5 always-run phases (BUILT-IN-DARK + BUILT-IN-LIGHT + PLAIN-MODE-ZERO-COLOR + DEPTH-AUTO-DETECT + YAML-OVERRIDE) with positive byte evidence
- [ ] P1-F20-T09 — Feature 20 close-out + push 4 remotes non-force — **PHASE 1 OF CLI-AGENT FUSION PROGRAMME COMPLETE**

Task 1: Bootstrap

Append F20 evidence section header (spec `0e97afa`), update PROGRESS current focus to F20 (replacing F19 close-out's "F20 next candidate" pointer), insert F20 task list (9 items) after F19's, ensure `06_phase_1_evidence.md` has an F20 anchor. Verify `gopkg.in/yaml.v3` is currently in `helix_code/go.mod` (likely indirect; sanity check before T05 imports it).

```
cd HelixCode && grep "yaml.v3" go.mod  # confirm presence (indirect or direct)
```

Commit: `docs(P1-F20-T01): bootstrap Phase 1 / Feature 20 evidence + advance PROGRESS.`

Task 2: types.go (TDD)

Files: new `helix_code/internal/theme/types.go`, new `helix_code/internal/theme/types_test.go`.

Define: - `Role int` enum + 5 values (`RoleInfo` / `RoleWarn` / `RoleError` / `RoleHighlight` / `RoleDim`) + private `numRoles` sentinel for array sizing. - `Role.String()` returning lowercase names; `ParseRole(s string) (Role, error)` returning `ErrInvalidRole` for unknown. - `ColorDepth int` enum + 4 values (`DepthOff` / `DepthANSI16` / `DepthANSI256` / `DepthTruecolor`) + `ColorDepth.String()`. - `ThemeName string` type + 3 constants (`ThemeDarkName` / `ThemeLightName` / `ThemeNoneName`). - `Color struct { Open string }`. - `Reset` const (`"\x1b[0m"`). - `Theme struct { Name ThemeName; ANSI16 [numRoles]Color; ANSI256 [numRoles]Color; Truecolor [numRoles]Color }`. - Error sentinels (`ErrUnknownTheme`, `ErrInvalidYAML`, `ErrInvalidRole`).

Failing tests FIRST:

```
func TestRole_String_LowercaseNames(t *testing.T) {
    require.Equal(t, "info", RoleInfo.String())
    require.Equal(t, "warn", RoleWarn.String())
    require.Equal(t, "error", RoleError.String())
    require.Equal(t, "highlight", RoleHighlight.String())
    require.Equal(t, "dim", RoleDim.String())
}

func TestRole_ParseRole_OK_ForAll5(t *testing.T) {
    for name, want := range map[string]Role{
        "info": RoleInfo, "warn": RoleWarn, "error": RoleError,
        "highlight": RoleHighlight, "dim": RoleDim,
    } {
        got, err := ParseRole(name)
        require.NoError(t, err); require.Equal(t, want, got)
    }
}

func TestRole_ParseRole_Unknown_Err(t *testing.T) {
    _, err := ParseRole("bogus")
    require.ErrorIs(t, err, ErrInvalidRole)
}

func TestColorDepth_String_AllFourValues(t *testing.T) {
    require.Equal(t, "off",       DepthOff.String())
    require.Equal(t, "ansi16",    DepthANSI16.String())
    require.Equal(t, "ansi256",   DepthANSI256.String())
    require.Equal(t, "truecolor", DepthTruecolor.String())
}

func TestThemeName_StableConstants(t *testing.T) {
```

```
    require.Equal(t, ThemeName("dark"),  ThemeDarkName)
    require.Equal(t, ThemeName("light"), ThemeLightName)
    require.Equal(t, ThemeName("none"),  ThemeNoneName)
}

func TestColor_OpenZeroValueIsEmpty(t *testing.T) {
    require.Empty(t, Color{}.Open)
}

func TestReset_LiteralByteSequence(t *testing.T) {
    require.Equal(t, "\x1b[0m", Reset)
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{ErrUnknownTheme, ErrInvalidYAML,
        ErrInvalidRole} {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}
```

Subject: `feat(P1-F20-T02): theme types - Role + Color + ColorDepth + Theme + sentinels + Reset const.`

Task 3: builtin.go (TDD; pin §3.4 byte tables)

Files: new `helix_code/internal/theme/builtin.go`, new `helix_code/internal/theme/builtin_test.go`.

`builtin.go`:

```
var ThemeDark = Theme{
    Name: ThemeDarkName,
    ANSI16: [numRoles]Color{
        RoleInfo:      {Open: "\x1b[37m"},
        RoleWarn:      {Open: "\x1b[33m"},
        RoleError:     {Open: "\x1b[31m"},
        RoleHighlight: {Open: "\x1b[36m"},
        RoleDim:       {Open: "\x1b[90m"},
    },
    ANSI256: [numRoles]Color{
        RoleInfo:      {Open: "\x1b[38;5;250m"},
        RoleWarn:      {Open: "\x1b[38;5;214m"},
        RoleError:     {Open: "\x1b[38;5;196m"},
        RoleHighlight: {Open: "\x1b[38;5;51m"},
        RoleDim:       {Open: "\x1b[38;5;243m"},
    },
    Truecolor: [numRoles]Color{
```

```
        RoleInfo:      {Open: "\x1b[38;2;220;220;220m"},
        RoleWarn:      {Open: "\x1b[38;2;255;176;0m"},
        RoleError:     {Open: "\x1b[38;2;255;64;64m"},
        RoleHighlight: {Open: "\x1b[38;2;0;200;220m"},
        RoleDim:       {Open: "\x1b[38;2;128;128;128m"},
    },
}

var ThemeLight = Theme{ /* per spec §3.4 */ }
var ThemeNone  = Theme{ Name: ThemeNoneName /* all Color{} zero
        values */ }

func BuiltIn() map[ThemeName]Theme {
    return map[ThemeName]Theme{
        ThemeDarkName:  ThemeDark,
        ThemeLightName: ThemeLight,
        ThemeNoneName:  ThemeNone,
    }
}
```

Failing tests FIRST (pin every byte from spec §3.4):

```
func TestBuiltIn_DarkAllRolesAllDepths_BytesPinned(t *testing.T)
        {
    // ANSI16
    require.Equal(t, "\x1b[37m",
        ThemeDark.ANSI16[RoleInfo].Open)
    require.Equal(t, "\x1b[33m",
        ThemeDark.ANSI16[RoleWarn].Open)
    require.Equal(t, "\x1b[31m",
        ThemeDark.ANSI16[RoleError].Open)
    require.Equal(t, "\x1b[36m",
        ThemeDark.ANSI16[RoleHighlight].Open)
    require.Equal(t, "\x1b[90m",  ThemeDark.ANSI16[RoleDim].Open)
    // ANSI256
    require.Equal(t, "\x1b[38;5;250m",
        ThemeDark.ANSI256[RoleInfo].Open)
    require.Equal(t, "\x1b[38;5;214m",
        ThemeDark.ANSI256[RoleWarn].Open)
    require.Equal(t, "\x1b[38;5;196m",
        ThemeDark.ANSI256[RoleError].Open)
    require.Equal(t, "\x1b[38;5;51m",
        ThemeDark.ANSI256[RoleHighlight].Open)
    require.Equal(t, "\x1b[38;5;243m",
        ThemeDark.ANSI256[RoleDim].Open)
    // Truecolor
    require.Equal(t, "\x1b[38;2;220;220;220m",
        ThemeDark.Truecolor[RoleInfo].Open)
```

```
    require.Equal(t, "\x1b[38;2;255;176;0m",
        ThemeDark.Truecolor[RoleWarn].Open)
    require.Equal(t, "\x1b[38;2;255;64;64m",
        ThemeDark.Truecolor[RoleError].Open)
    require.Equal(t, "\x1b[38;2;0;200;220m",
        ThemeDark.Truecolor[RoleHighlight].Open)
    require.Equal(t, "\x1b[38;2;128;128;128m",
        ThemeDark.Truecolor[RoleDim].Open)
}

func TestBuiltIn_LightAllRolesAllDepths_BytesPinned(t
        *testing.T) { /* spec §3.4 light table */ }

func TestBuiltIn_NoneAllOpenAreEmpty_AllDepthsAllRoles(t
        *testing.T) {
    for r := Role(0); r < numRoles; r++ {
        require.Empty(t, ThemeNone.ANSI16[r].Open,
        "ANSI16[%v]", r)
        require.Empty(t, ThemeNone.ANSI256[r].Open,
        "ANSI256[%v]", r)
        require.Empty(t, ThemeNone.Truecolor[r].Open,
        "Truecolor[%v]", r)
    }
}

func TestBuiltIn_RegistryReturnsThreeThemes(t *testing.T) {
    m := BuiltIn()
    require.Len(t, m, 3)
    require.Contains(t, m, ThemeDarkName)
    require.Contains(t, m, ThemeLightName)
    require.Contains(t, m, ThemeNoneName)
}

func TestBuiltIn_NamesMatchConstants(t *testing.T) {
    require.Equal(t, ThemeDarkName,  ThemeDark.Name)
    require.Equal(t, ThemeLightName, ThemeLight.Name)
    require.Equal(t, ThemeNoneName,  ThemeNone.Name)
}
```

Subject: `feat(P1-F20-T03): builtin themes dark/light/none with pinned byte tables.`

Task 4: detect.go (TDD; table-drive every branch)

Files: new `helix_code/internal/theme/detect.go`, new `helix_code/internal/theme/detect_test.go`.

`detect.go`:

```go
type ResolvedSource string

const (
    SourceEnv       ResolvedSource = "env"
    SourceColorFGBG ResolvedSource = "colorfgbg"
    SourceDefault   ResolvedSource = "default"
)

// DetectThemeName resolves the active theme name per spec §3.6
        ladder.
// env is the env-var lookup closure; passing os.Getenv yields
        production
// behaviour. Tests inject a fake closure to exercise every
        branch.
func DetectThemeName(env func(string) string) (ThemeName,
        ResolvedSource) {
    if v := env("HELIXCODE_THEME"); v != "" {
        // matched against built-ins OR YAML names — names are
        opaque, so
        // the loader (Task 5) is the place that disambiguates
        "unknown".
        // detect.go just returns the raw name + SourceEnv so the
        loader
        // can warn-and-fall-through if the name doesn't match
        anything.
        return ThemeName(v), SourceEnv
    }
    if v := env("COLORFGBG"); v != "" {
        if name, ok := parseColorFGBG(v); ok {
            return name, SourceColorFGBG
        }
    }
    return ThemeDarkName, SourceDefault
}

// DetectColorDepth resolves the active color depth per spec
        §3.6.
func DetectColorDepth(env func(string) string) ColorDepth {
    switch env("COLORTERM") {
    case "truecolor", "24bit":
        return DepthTruecolor
    }
    term := env("TERM")
    if term == "" || term == "dumb" {
        return DepthOff
    }
    if strings.HasSuffix(term, "-256color") {
        return DepthANSI256
```

```
    }
    return DepthANSI16
}

// parseColorFGBG returns the inferred theme name from the bg
        index of the
// "fg;bg" pair. Returns (_, false) for unparseable values
        (silent — opportunistic
// signal per spec §5.1).
func parseColorFGBG(v string) (ThemeName, bool) {
    parts := strings.Split(v, ";")
    if len(parts) < 2 { return "", false }
    bg, err := strconv.Atoi(parts[len(parts)-1]) // tolerate
        "fg;_;bg" and "fg;bg"
    if err != nil { return "", false }
    // Dark backgrounds: 0,1,2,3,4,5,6,8 (per xterm convention)
    // Light backgrounds: 7,9..15
    if bg == 7 || (bg >= 9 && bg <= 15) {
        return ThemeLightName, true
    }
    if bg >= 0 && bg <= 8 {
        return ThemeDarkName, true
    }
    return "", false
}
```

Failing tests FIRST (table-driven):

```
func fakeEnv(m map[string]string) func(string) string {
    return func(k string) string { return m[k] }
}

func TestDetectThemeName_EnvSet_ReturnsEnv(t *testing.T) {
    name, src :=
        DetectThemeName(fakeEnv(map[string]string{"HELIXCODE_THEME":
        "light"}))
    require.Equal(t, ThemeName("light"), name)
    require.Equal(t, SourceEnv, src)
}

func TestDetectThemeName_NoEnv_ColorFGBGDarkBg_ReturnsDark(t
        *testing.T) {
    name, src :=
        DetectThemeName(fakeEnv(map[string]string{"COLORFGBG":
        "15;0"}))
    require.Equal(t, ThemeDarkName, name)
    require.Equal(t, SourceColorFGBG, src)
}
```

```
func TestDetectThemeName_NoEnv_ColorFGBGLightBg_ReturnsLight(t
        *testing.T) {
    name, src :=
        DetectThemeName(fakeEnv(map[string]string{"COLORFGBG":
        "0;15"}))
    require.Equal(t, ThemeLightName, name)
    require.Equal(t, SourceColorFGBG, src)
}

func TestDetectThemeName_NoEnv_ColorFGBGUnparseable_FallsThrough(t
        *testing.T) {
    name, src :=
        DetectThemeName(fakeEnv(map[string]string{"COLORFGBG":
        "garbage"}))
    require.Equal(t, ThemeDarkName, name)
    require.Equal(t, SourceDefault, src)
}

func TestDetectThemeName_NoSignals_ReturnsDarkDefault(t
        *testing.T) {
    name, src := DetectThemeName(fakeEnv(map[string]string{}))
    require.Equal(t, ThemeDarkName, name)
    require.Equal(t, SourceDefault, src)
}

func TestDetectColorDepth_ColorTermTruecolor(t *testing.T) {
    require.Equal(t, DepthTruecolor,
        DetectColorDepth(fakeEnv(map[string]string{"COLORTERM":
        "truecolor"})))
}
func TestDetectColorDepth_ColorTerm24bit(t *testing.T) {
    require.Equal(t, DepthTruecolor,
        DetectColorDepth(fakeEnv(map[string]string{"COLORTERM":
        "24bit"})))
}
func TestDetectColorDepth_TermXterm256(t *testing.T) {
    require.Equal(t, DepthANSI256,
        DetectColorDepth(fakeEnv(map[string]string{"TERM":
        "xterm-256color"})))
}
func TestDetectColorDepth_TermXtermPlain(t *testing.T) {
    require.Equal(t, DepthANSI16,
        DetectColorDepth(fakeEnv(map[string]string{"TERM":
        "xterm"})))
}
func TestDetectColorDepth_TermDumb_Off(t *testing.T) {
```

```
    require.Equal(t, DepthOff,
        DetectColorDepth(fakeEnv(map[string]string{"TERM":
        "dumb"})))
}
func TestDetectColorDepth_TermUnset_Off(t *testing.T) {
    require.Equal(t, DepthOff,
        DetectColorDepth(fakeEnv(map[string]string{})))
}
func TestDetectColorDepth_ColorTermBeatsTERM(t *testing.T) {
    // COLORTERM=truecolor wins even if TERM=dumb
    require.Equal(t, DepthTruecolor,
        DetectColorDepth(fakeEnv(map[string]string{
        "COLORTERM": "truecolor", "TERM": "dumb",
    })))
}
```

Subject: `feat(P1-F20-T04): theme name + color depth detection from injected env closure.`

Task 5: loader.go (TDD; YAML merge into dark baseline)

Files: new `helix_code/internal/theme/loader.go`, new `helix_code/internal/theme/loader_test.go`. **Modify** `helix_code/go.mod` to promote `gopkg.in/yaml.v3` from indirect to direct (verify with `go mod tidy` produces no new `go.sum` entries).

`loader.go`:

```
type LoaderOptions struct {
    Env        func(string) string // default os.Getenv
    ConfigDir  string              // default $XDG_CONFIG_HOME/
        helixcode (or $HOME/.config/helixcode)
    Filesystem
        fs.FS                // default os.DirFS — injected for
        tests
}

type Loader struct {
    env       func(string) string
    configDir string
    fsys      fs.FS
}

func NewLoader(opts LoaderOptions) *Loader { /* fill defaults
        */ }

func (l *Loader) Load() (*Styler, ResolvedSource, error) {
    name, source := DetectThemeName(l.env)
```

```
    depth        := DetectColorDepth(l.env)

    builtIns := BuiltIn()
    yamlThemes, err := l.loadYAML()  // returns
        map[ThemeName]Theme; nil error if file missing
    if err != nil {
        // wrap with ErrInvalidYAML; warn to stderr; proceed with
        built-ins only.
        fmt.Fprintf(os.Stderr, "theme: %s: falling back to built-
        ins\n", err)
        yamlThemes = nil
    }

    var theme Theme
    if t, ok := yamlThemes[name]; ok {
        theme = t
    } else if t, ok := builtIns[name]; ok {
        theme = t
    } else {
        // unknown name: warn + fall back to dark default
        fmt.Fprintf(os.Stderr, "theme: HELIXCODE_THEME=%q not
        found; falling back to %q\n", name, ThemeDarkName)
        theme = builtIns[ThemeDarkName]
        source = SourceDefault
        name = ThemeDarkName
    }

    s := &Styler{theme: theme, depth: depth, source: source}
    return s, source, nil
}

// loadYAML reads $ConfigDir/theme.yaml if present; returns
        parsed themes
// merged into the dark built-in baseline (slot-by-slot overlay).
        Missing file
// returns (nil, nil); parse error returns (nil, ErrInvalidYAML
        wrapped).
func (l *Loader) loadYAML() (map[ThemeName]Theme, error)
        { /* ... */ }

type Styler struct {
    theme  Theme
    depth  ColorDepth
    source ResolvedSource
}

func (s *Styler) Stylize(role Role, text string) string {
    if s.depth == DepthOff {
```

```
        return text
    }
    if role < 0 || role >= numRoles {
        return text
    }
    var c Color
    switch s.depth {
    case DepthANSI16:    c = s.theme.ANSI16[role]
    case DepthANSI256:   c = s.theme.ANSI256[role]
    case DepthTruecolor: c = s.theme.Truecolor[role]
    }
    if c.Open == "" {
        return text
    }
    return c.Open + text + Reset
}

func (s *Styler) Theme()  Theme           { return s.theme }
func (s *Styler) Depth()  ColorDepth      { return s.depth }
func (s *Styler) Source() ResolvedSource  { return s.source }

// WithDepth returns a NEW Styler with the given depth; the
        original is
// immutable (zero-allocation pointer to value-type copy).
func (s *Styler) WithDepth(d ColorDepth) *Styler {
    return &Styler{theme: s.theme, depth: d, source: s.source}
}
```

Implementation notes: - YAML merge: parse the file into a `map[string]map[string]map[string]string` (theme name → depth key → role key → open code), then for each theme name overlay onto a copy of `ThemeDark` (per spec §3.7 v1 simplification; F20.5 may add `base:` field). Validate every role/depth key + every open code starts with `\x1b[` and ends with `m` — surface unknown keys + malformed open codes as `ErrInvalidYAML`-wrapped errors. - `loadYAML` uses `fs.ReadFile(l.fsys, "theme.yaml")` for testability (a tempdir-backed `os.DirFS` in production; an `fstest.MapFS` in unit tests). - `Stylize` zero-alloc fast path for `DepthOff` returns the input string by value (Go strings are reference types; no copy). - The detector picks `name` from env (which may be a YAML-only name); the loader checks YAML themes first, then built-ins, then warn-fallback to dark.

Failing tests FIRST:

```
func TestLoader_NoYAMLFile_BuiltInsOnly(t *testing.T) {
    loader := NewLoader(LoaderOptions{
        Env: fakeEnv(map[string]string{"HELIXCODE_THEME":
        "dark", "COLORTERM": "truecolor"}),
        Filesystem: fstest.MapFS{}, // empty
    })
    s, src, err := loader.Load()
    require.NoError(t, err)
```

```
    require.Equal(t, SourceEnv, src)
    require.Equal(t, ThemeDarkName, s.Theme().Name)
    require.Equal(t, DepthTruecolor, s.Depth())
}

func TestLoader_YAMLEmpty_NoError(t *testing.T) {
    loader := NewLoader(LoaderOptions{
        Env: fakeEnv(map[string]string{}),
        Filesystem: fstest.MapFS{"theme.yaml":
        &fstest.MapFile{Data: []byte("")}},
    })
    _, _, err := loader.Load()
    require.NoError(t, err)
}

func TestLoader_YAMLParseErr_FallsBackToBuiltIns(t *testing.T) {
    loader := NewLoader(LoaderOptions{
        Env: fakeEnv(map[string]string{}),
        Filesystem: fstest.MapFS{"theme.yaml":
        &fstest.MapFile{Data: []byte("not: valid: yaml: [\n")}},
    })
    s, _, err := loader.Load()
    // load doesn't return the error — it warns and falls back
        per §5.1.
    require.NoError(t, err)
    require.Equal(t, ThemeDarkName, s.Theme().Name)
}

func TestLoader_YAMLSchema_UnknownRole_ErrInvalidYAML(t
        *testing.T) {
    yaml := []byte("themes:\n  custom:\n    ansi16:\n
        infoo: \"\\x1b[31m\"\n")
    loader := NewLoader(LoaderOptions{
        Env: fakeEnv(map[string]string{"HELIXCODE_THEME":
        "custom"}),
        Filesystem: fstest.MapFS{"theme.yaml":
        &fstest.MapFile{Data: yaml}},
    })
    // schema violation surfaces via the loader's WARN-and-
        fallback path; the
    // public Load returns no error but the Styler is the dark
        built-in.
    s, _, err := loader.Load()
    require.NoError(t, err)
    require.Equal(t, ThemeDarkName, s.Theme().Name) // fell back
        per §3.7
}
```

```
func TestLoader_YAMLOverridesBuiltInDark_PerSlot(t *testing.T) {
    yaml := []byte(`themes:
  custom:
    truecolor:
      error: "\x1b[38;2;220;50;47m"
`)
    loader := NewLoader(LoaderOptions{
        Env: fakeEnv(map[string]string{"HELIXCODE_THEME":
        "custom", "COLORTERM": "truecolor"}),
        Filesystem: fstest.MapFS{"theme.yaml":
        &fstest.MapFile{Data: yaml}},
    })
    s, _, err := loader.Load()
    require.NoError(t, err)
    require.Equal(t, ThemeName("custom"), s.Theme().Name)
    require.Equal(t, "\x1b[38;2;220;50;47m",
        s.Theme().Truecolor[RoleError].Open)
    // Un-mentioned slot inherits from dark baseline:
    require.Equal(t, "\x1b[38;2;220;220;220m",
        s.Theme().Truecolor[RoleInfo].Open)
}

func TestStyler_Stylize_DepthOff_IdentityForAllRoles(t
        *testing.T) {
    s := &Styler{theme: ThemeDark, depth: DepthOff}
    for r := Role(0); r < numRoles; r++ {
        require.Equal(t, "hello", s.Stylize(r, "hello"))
    }
}

func TestStyler_Stylize_DepthANSI16_DarkInfo_BytesPinned(t
        *testing.T) {
    s := &Styler{theme: ThemeDark, depth: DepthANSI16}
    require.Equal(t, "\x1b[37mhello\x1b[0m", s.Stylize(RoleInfo,
        "hello"))
}

func TestStyler_Stylize_DepthANSI256_DarkError_BytesPinned(t
        *testing.T) {
    s := &Styler{theme: ThemeDark, depth: DepthANSI256}
    require.Equal(t, "\x1b[38;5;196mboom\x1b[0m",
        s.Stylize(RoleError, "boom"))
}

func TestStyler_Stylize_DepthTruecolor_DarkHighlight_BytesPinned(t
        *testing.T) {
    s := &Styler{theme: ThemeDark, depth: DepthTruecolor}
```

```
    require.Equal(t, "\x1b[38;2;0;200;220mfoo\x1b[0m",
        s.Stylize(RoleHighlight, "foo"))
}

func TestStyler_Stylize_NoneAllDepths_AllRolesIdentity(t
        *testing.T) {
    for _, depth := range []ColorDepth{DepthANSI16,
        DepthANSI256, DepthTruecolor} {
        s := &Styler{theme: ThemeNone, depth: depth}
        for r := Role(0); r < numRoles; r++ {
            require.Equal(t, "x", s.Stylize(r, "x"))
        }
    }
}

func TestStyler_WithDepth_ReturnsNewStylerCorrectDepth(t
        *testing.T) {
    s1 := &Styler{theme: ThemeDark, depth: DepthTruecolor}
    s2 := s1.WithDepth(DepthOff)
    require.Equal(t, DepthOff, s2.Depth())
    require.Equal(t, DepthTruecolor, s1.Depth(),
        "original Styler must be unchanged")
}

func TestNoDirectPaletteAccess_BypassesStyler_SourceScan(t
        *testing.T) {
    // anti-bluff (c) — no call site outside internal/theme
        should index the
    // theme array directly. Greps cmd/cli/ + internal/commands/
        theme_command.go.
    paths := []string{"../../cmd/cli", "../../internal/commands/
        theme_command.go"}
    pattern := regexp.MustCompile(`theme\.Theme(Dark|Light|None)
        \.(Truecolor|ANSI256|ANSI16)\[`)
    // walk paths; assert pattern.Match returns no hits in
        any .go file
    // (full impl in test file)
}
```

After T05 implementation:

```
cd HelixCode && go mod tidy
git diff go.mod   # should ONLY show yaml.v3 indirect→direct line
git diff go.sum   # should be empty
```

Subject: `feat(P1-F20-T05): theme loader + Styler with YAML merge into dark baseline (TDD).`

Task 6: Wire Styler into F18 (TDD)

Files: modify `helix_code/cmd/cli/main.go`. Add new test `helix_code/cmd/cli/main_theme_wire_test.go` if a focused unit harness fits, otherwise the integration test in T07 carries the byte assertions.

Three small additions in `cmd/cli/main.go`:

1. Construct `theme.Loader` adjacent to F18's renderer:

```
themeLoader := theme.NewLoader(theme.LoaderOptions{})
styler, _, err := themeLoader.Load()
if err != nil {
    log.Printf("theme: %v; using built-ins", err)
}
if c.renderer.Mode() == render.ModePlain {
    styler = styler.WithDepth(theme.DepthOff)
}
c.styler = styler
c.themeLoader = themeLoader
```

2. Wire the Styler into `handleGenerate`'s non-stream branch (the `RenderTextBlock` call added in F18-T08). Style the response body with `RoleInfo`; style any `err.Error()` output emitted on the same path with `RoleError`:

```
styled := c.styler.Stylize(theme.RoleInfo, resp.Text)
if err := render.RenderTextBlock(c.renderer, "", styled);
        err != nil { /* ... */ }
```

3. Add the `c.styler` and `c.themeLoader` fields to the `*CLI` struct.

Failing tests FIRST (focus on the byte-evidence at the wire point):

```
func TestCLIWire_HandleGenerate_FancyRenderer_StyledResponseHasANSIBytes(t
        *testing.T) {
    // wire a real ansiRenderer (via FactoryOptions IsTTY=true) +
        real Styler
    // (dark + Truecolor) into a *CLI; call handleGenerate's non-
        stream branch
    // with a stub provider that returns "hello" → captured
        stdout MUST contain
    //   \x1b[38;2;220;220;220mhello\x1b[0m
    // (one open + literal + one reset).
}

func TestCLIWire_PlainRenderer_StylerOverriddenToOff_NoANSIInOutput(t
        *testing.T) {
    // wire a real plainRenderer + Styler force-overridden to
        DepthOff via
```

```
    // WithDepth; assert captured stdout contains "hello" but NO
        0x1b byte.
}
```

If the existing `handleGenerate` test seam in `main.go` is too tangled for a focused unit test, defer the byte-assertions to T07's integration test (`tests/integration/theme_test.go::TestTheme_Integration_FancyRenderer_DarkInfoEmitsANSI`). Either way, the byte-evidence is asserted via the production code path before T08's Challenge.

Subject: `feat(P1-F20-T06): wire theme.Styler into handleGenerate via F18 RenderTextBlock.`

Task 7: /theme slash command + integration test (TDD)

Files: new `helix_code/internal/commands/theme_command.go`, new `helix_code/internal/commands/theme_command_test.go`, modify `helix_code/cmd/cli/main.go` (one line: register the command), new `helix_code/tests/integration/theme_test.go` (`//go:build integration`).

`theme_command.go`:

```
type ThemeCommand struct {
    styler *theme.Styler  // active styler
    loader *theme.Loader  // for /theme list discovery (re-reads
        YAML names)
}

func NewThemeCommand(s *theme.Styler, l *theme.Loader)
        *ThemeCommand {
    return &ThemeCommand{styler: s, loader: l}
}

func (c *ThemeCommand) Name() string         { return "theme" }
func (c *ThemeCommand) Aliases() []string    { return nil }
func (c *ThemeCommand) Description() string {
    return "Show the active theme, list available themes, or
        preview a theme by name."
}
func (c *ThemeCommand) Usage() string        { return "/theme
        [status|list|show <name>]" }

func (c *ThemeCommand) Execute(ctx context.Context, cc
        *CommandContext) (*CommandResult, error) {
    args := cc.Args
    sub := "status"
    if len(args) > 0 { sub = args[0] }
    switch sub {
    case "status":
        return c.status(), nil
```

```
    case "list":
        return c.list(), nil
    case "show":
        if len(args) < 2 { return nil, fmt.Errorf("/theme show
        <name>") }
        return c.show(args[1])
    default:
        return nil, fmt.Errorf("/theme: unknown subcommand %q
        (want status|list|show)", sub)
    }
}

// status prints "active=<name> depth=<depth> source=<source>"
        three-line table.
func (c *ThemeCommand) status() *CommandResult { /* ... */ }

// list prints all available theme names: built-ins + YAML;
        active one
// prefixed with "* " (RoleHighlight).
func (c *ThemeCommand) list() *CommandResult { /* ... */ }

// show looks up <name> in built-ins + YAML; renders the 5 sample
        lines using
// the named theme's palette at the active depth (per spec §4.3).
func (c *ThemeCommand) show(name string) (*CommandResult, error)
        { /* ... */ }
```

Failing tests FIRST:

```
func TestThemeCommand_Name_IsTheme(t *testing.T) {
    require.Equal(t, "theme", NewThemeCommand(nil, nil).Name())
}

func TestThemeCommand_Status_PrintsActiveNameAndDepthAndSource(t
        *testing.T) {
    s := &theme.Styler{ /* ... */ }  // dark + Truecolor +
        SourceEnv
    cmd := NewThemeCommand(s, nil)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "active=dark")
    require.Contains(t, res.Output, "depth=truecolor")
    require.Contains(t, res.Output, "source=env")
}

func TestThemeCommand_List_IncludesAllBuiltIns(t *testing.T) { /
        * ... */ }
```

```
func TestThemeCommand_List_IncludesYAMLOverrides(t *testing.T)
        { /* ... */ }
func TestThemeCommand_Show_KnownName_RendersFiveRoles(t
        *testing.T) { /* ... */ }
func TestThemeCommand_Show_UnknownName_Err(t *testing.T) {
    cmd := NewThemeCommand(/* ... */)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"show", "nope"}})
    require.Error(t, err)
}
func TestThemeCommand_Show_UsesNamedThemePalette_NotActive(t
        *testing.T) {
    // active styler is dark+Truecolor; /theme show light
        produces light's
    // Truecolor codes in the rendered samples.
}
```

`main.go` registration:

```
if regErr :=
        c.commandRegistry.Register(commands.NewThemeCommand(c.styler,
        c.themeLoader)); regErr != nil {
    log.Printf("theme: register slash command failed: %v",
        regErr)
}
```

Integration tests (`tests/integration/theme_test.go` — `//go:build integration`; ALWAYS-runs):

```
//go:build integration

package integration

func TestTheme_Integration_FancyRenderer_DarkInfoEmitsANSI(t
        *testing.T) {
    // wire real ansiRenderer (FactoryOptions IsTTY=true on
        bytes.Buffer) +
    // real Styler (dark, Truecolor) → RenderTextBlock with
        Stylize-wrapped text
    // → assert captured contains \x1b[38;2;220;220;220m AND
        "hello" AND \x1b[0m.
}

func TestTheme_Integration_PlainRenderer_StylerForcedOff_ZeroANSI(t
        *testing.T) {
    // wire real plainRenderer + Styler force-overridden to
        DepthOff →
    // RenderTextBlock with Stylize-wrapped text → assert
        bytes.IndexByte(out, 0x1b) == -1.
```

```
}

func TestTheme_Integration_DepthAutoDetect_AllFourBranches(t
        *testing.T) {
    // 4 sub-cases via injected Env closure on Loader.
}

func TestTheme_Integration_YAMLOverride_CustomNameLoads(t
        *testing.T) {
    // os.MkdirTemp + write theme.yaml + LoaderOptions.ConfigDir
        injection.
}
```

Subject: `feat(P1-F20-T07): /theme slash command (status/list/show) + main.go wiring + integration test.`

Task 8: Challenge harness (5 always-run phases, positive byte evidence)

Files: new `helix_code/tests/integration/cmd/p1f20_challenge/main.go`, new `challenges/p1-f20-theme-system/CHALLENGE.md`, new `challenges/p1-f20-theme-system/run.sh`.

Harness phases (per spec §6.3):

1. **PHASE-A: BUILT-IN-DARK (always runs)** — real `ansiRenderer` + Styler(dark, Truecolor) → write 5 styled tokens via `RenderTextBlock(r, "", styler.Stylize(roleI, tokenI))` for the 5 roles → assert (i) captured contains `\x1b[38;2;220;220;220m` (Info Truecolor open), (ii) captured contains all 5 role-colored byte sequences, (iii) `bytes.Count(captured, []byte("\x1b[")) == 2 * 5` (one open + one reset per token).
2. **PHASE-B: BUILT-IN-LIGHT (always runs)** — same wiring with `light` theme + Truecolor → assert byte offsets put open < literal < reset for each token.
3. **PHASE-C: PLAIN-MODE-ZERO-COLOR (always runs)** — real `plainRenderer` + Styler(dark, Truecolor) force-overridden to `DepthOff` → write 100 styled tokens → assert `bytes.IndexByte(captured, 0x1b) == -1` AND captured contains all 100 token literals.
4. **PHASE-D: DEPTH-AUTO-DETECT (always runs)** — 6 representative env combinations via `Loader` with injected `Env` closure → for each, assert resolved `(name, source, depth)` tuple matches expected:
    - `{COLORTERM:truecolor, TERM:xterm-256color}` → `(dark, default, truecolor)`
    - `{TERM:xterm-256color}` → `(dark, default, ansi256)`
    - `{TERM:xterm}` → `(dark, default, ansi16)`
    - `{TERM:dumb}` → `(dark, default, off)`
    - `{}` → `(dark, default, off)`
    - `{HELIXCODE_THEME:nope, COLORFGBG:15;0}` → `(dark, colorfgbg, off)` *(unknown env name + colorfgbg dark bg → fall through to dark via SourceColorFGBG)*
5. **PHASE-E: YAML-OVERRIDE (always runs)** — `os.MkdirTemp` + write `theme.yaml` containing `custom-solarized` palette + Env injection

(HELIXCODE_THEME=custom-solarized, COLORTERM=truecolor) + Loader.Load() → assert (i) Styler.Theme().Name == "custom-solarized", (ii) Stylize(RoleError, "x") == "\x1b[38;2;220;50;47m" + "x" + "\x1b[0m", (iii) un-mentioned slot (e.g. RoleHighlight at ANSI16) inherits from dark (\x1b[36m).

Output skeleton (verbatim per spec §6.3) ends with:

```
SUMMARY: PHASE-A=6/6 PASS; PHASE-B=4/4 PASS; PHASE-C=4/4 PASS; PHASE-D=6/6
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into 06_phase_1_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

challenges/p1-f20-theme-system/run.sh mirrors F18/F19 structure: cd HelixCode && go run ./tests/integration/cmd/p1f20_challenge/main.go.

Subject: feat(P1-F20-T08): challenge with 5 always-run phases + byte-count + zero-byte + depth-resolution + YAML-merge positive evidence.

Task 9: Close-out + push — PHASE 1 PROGRAMME COMPLETE

Tick all 9 items in PROGRESS, advance PROGRESS focus from F20 to **"Phase 1 of CLI-Agent Fusion programme COMPLETE"** (with a one-paragraph synthesis summarising the 20 features shipped + key invariants). Run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/theme internal/commands/theme_command.go && echo
        BLUFF || echo clean
go test -count=1 ./internal/theme/...
go test -count=1 ./internal/commands/...
go test -count=1 -tags=integration ./tests/integration/...
go mod tidy
git diff --exit-code go.sum
        # MUST be no-op (yaml.v3 was already in go.sum)
git diff go.mod  # may show ONLY the yaml.v3 indirect→direct
        promotion line
```

Cross-compile check (matches F18/F19):

```
cd HelixCode && GOOS=linux GOARCH=amd64 go build -o /tmp/
        helixcode-linux-amd64 ./cmd/server
ls -la /tmp/helixcode-linux-amd64  # confirm produced binary
```

Commit chore(P1-F20-T09): close out feature 20 — Phase 1 of CLI-Agent Fusion programme COMPLETE. Push 4 remotes non-force (origin, helixdev,

`vasic-digital`, `gitlab` per programme conventions). Request explicit user authorization at this step (CONST-043).

PROGRESS.md milestone entry (verbatim):

```
- 2026-05-06 — Feature 20 (Theme System) closed. 9 task commits (T01 ..., 
  Real, end-to-end 5-role semantic theme system for the HelixCode CLI agen
  dark/light/none + optional YAML override at $XDG_CONFIG_HOME/helixcode/t
  env-var (HELIXCODE_THEME) + $COLORFGBG fallback for theme name resolutio
  $COLORTERM/$TERM auto-detect for color depth (Truecolor/ANSI256/ANSI16/O
  /theme slash command (status/list/show <name>); zero new external deps 
  (yaml.v3 promoted indirect→direct, already in go.sum). [5-phase Challeng
  evidence summary]. **Phase 1 of CLI-Agent Fusion programme COMPLETE — 20
  features shipped (F01..F20).**
```

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + sentinels (§3.3), T03 built-in palettes (§3.4 byte tables), T04 detection (§3.6), T05 loader + Styler + YAML merge (§3.3 Styler API + §3.7 YAML schema + §5.5 cross-depth gap), T06 wire into F18 (§2 wire-points + §4.1 startup), T07 /theme slash (§4.3) + integration tests (§6.2), T08 Challenge five phases (§5.2 + §6.3), T09 close-out (§9 + Phase 1 milestone).
2. **TDD:** every code task starts with failing tests. Built-in tests pin §3.4's tables byte-for-byte (no rounding, no recomputation — the literals in code MUST equal the literals in spec). Detection tests use a `func(string) string` closure (no `os.Setenv` ever — pure-function tests). Loader tests use `fstest.MapFS` for YAML inputs. Styler tests assert byte equality on the wrapped text. /theme command tests use a constructed Styler value type (no mocks). Integration tests wire the production `Loader` + `Styler` end-to-end through F18's real `ansiRenderer` and `plainRenderer`.
3. **Type consistency:** `Theme`, `Role`, `Color`, `ColorDepth`, `ThemeName`, `Styler`, `LoaderOptions`, `ResolvedSource`, error sentinels (`ErrUnknownTheme`, `ErrInvalidYAML`, `ErrInvalidRole`), constants (`Reset`, `numRoles`, `ThemeDarkName`, `ThemeLightName`, `ThemeNoneName`, `SourceEnv`, `SourceColorFGBG`, `SourceDefault`), command name `theme`, env var `HELIXCODE_THEME` — all match across spec §3.3 and plan T02–T07.
4. **Zero new external deps:** stdlib + existing testify/zap + `gopkg.in/yaml.v3` (indirect→direct promotion only; the module is already in `go.sum` via viper/cobra). `go mod tidy` after T05 produces ONLY the indirect→direct promotion line in `go.mod`; ZERO new entries in `go.sum`. T09's verification step asserts `git diff --exit-code go.sum` is no-op.
5. **Anti-bluff (§5.2):** Challenge has FIVE always-run phases. Every phase records positive evidence: byte counts (Phase A: exactly 2 ANSI sequences per token), byte content + ordering (Phase B: open < literal < reset by offset), zero-byte invariants (Phase C: `bytes.IndexByte == -1`), depth-resolution byte-equality (Phase D: 6 env combos), YAML-merge byte-equality (Phase E: custom + inherited slots). The four real-execution criteria — (a) production Styler emits expected bytes for each (role × depth) (Phase A/B); (b) plain mode + force-Off Styler produces zero `0x1b` (Phase C); (c) detection covers all four depth branches (Phase D); (d) YAML override merges into dark baseline correctly (Phase E) — each have dedicated unit + integration + Challenge assertions. Source-scan unit test ensures no call site bypasses the Styler with direct palette indexing. Byte-evidence mismatch is a hard Challenge failure.

6. **CONST-042:** theme YAML carries no secrets (color codes are non-sensitive); file mode 0644 is acceptable. Loader does NOT log file contents at any level. A unit test scans `internal/theme/*.go` for `logger\.\b(Info|Debug)\b.*\b(palette|color|open|reset)\b` matches and FAILS on any hit (defensive log discipline; consistent with F19's stance even though there are no actual secrets to leak here).
7. **CONST-043:** stays on `main`, non-force to all four remotes; explicit user authorization is requested at T09 before pushing.
8. **CONST-033:** F20 emits SGR (`\x1b[<n>m`) sequences only — no device-status-report, no terminal-state-mutation escapes. Spec §9 records this explicitly.
9. **Decorator pattern (Stylize wraps text) vs renderer extension — non-obvious call** (recorded in spec §2 trailer + §11 #1): keeps F18's `Renderer` interface unchanged; plain mode statically proves no-leak via `DepthOff`; tests pin byte sequences without instantiating a renderer; migration is incremental (un-migrated call sites still emit unstyled text correctly).
10. **Five roles, not three or seven — non-obvious call** (recorded in spec §11 #2): `Info` / `Warn` / `Error` / `Highlight` / `Dim` covers the 80% case. Three is too few (no accent for chosen choice / current model name); seven adds noise (`Success` overlaps `Info`, `Critical` overlaps `Error`).
11. **Per-Theme `[5]Color` × 3 depth tiers (no runtime cross-depth conversion) — non-obvious call** (recorded in spec §11 #3): a theme designer hand-picks codes per depth so a `Truecolor` palette is not auto-derived from ANSI16 (which would lose the designer's intent); cost is more YAML lines for a custom theme; benefit is faithful palette rendering.
12. `$COLORFGBG` **parsing is opportunistic, NOT configured — non-obvious call** (recorded in spec §11 #4): malformed values fall through silently (no warn) because terminal emulators set this inconsistently (xterm sets `15;0`, gnome-terminal sets `7;0`, alacritty does not set it at all); warn-on-unparseable would WARN-spam every alacritty user.
13. **Plain-mode forces Styler depth=Off via `WithDepth(DepthOff)` at startup — non-obvious call** (recorded in spec §11 #5): belt-and-suspenders against future regressions in the renderer or Styler. Even if `DetectColorDepth` returns Truecolor, plain-mode `Stylize` is identity. Test `TestStyler_Stylize_DepthOff_IdentityForAllRoles` pins this.
14. **YAML merge always starts from `dark` in v1 — non-obvious call** (recorded in spec §11 #6): no `base:` field. Merge semantics simple (overlay slot-by-slot); optional base-selection deferred to F20.5. v1 docs the limitation in §3.7.
15. `none` **theme is distinct from depth=Off — non-obvious call** (recorded in spec §11 #7): both produce identical bytes (`Stylize(role, x) == x`); the distinction matters for `/theme status` (source / depth reporting) and for tests that pin each path independently.
16. **Styler does NOT probe TTY itself — non-obvious call** (recorded in spec §11 #8): F18's renderer factory is the source of truth; F20's startup code reads `renderer.Mode()` and conditionally overrides the Styler. Two probes would risk drift.
17. `/theme show <name>` **previews the named theme's palette, NOT the active styler's — non-obvious call** (recorded in spec §11 #9): the user is asking "what does light look like"; the answer must use light's data. Test `TestThemeCommand_Show_UsesNamedThemePalette_NotActive` pins this.
18. **Phase 1 close-out:** F20 is the **final** Phase 1 feature. T09 commit subject + PROGRESS.md milestone entry both record "Phase 1 of CLI-Agent Fusion programme COMPLETE" alongside the F20 close-out summary. The 20 features shipped (F01..F20) span auto-compaction, permission rules, tool-result persistence, git-worktree agent isolation, hooks, MCP, background tasks, plan mode, slash commands, skills, session resume, multi-provider, LSP, sandboxed shell, subagent team, OpenTelemetry, smart edit, no-flicker rendering, ask-user, and theme system.